



Memory your compliance team can read.

A hosted long-term memory layer for AI applications — one that can always tell you what it knows about someone, where it learned it, and why it changed its mind.

PITCH & TECHNICAL BRIEF • 20 AUGUST 2026

PART ONE

The pitch

Why this exists, who needs it, and what makes it defensible.

Every AI product forgets, and the fixes are worse than the problem.

TODAY'S FIX 01

Stuff the whole history in

Send every past message with every request. Costs grow linearly, latency with them, and the model still misses the one line that mattered.

TODAY'S FIX 02

Dump it in a vector store

Embed the transcript and hope similarity finds it. Nothing decides what was worth keeping, so noise and signal rank alike.

TODAY'S FIX 03

Overwrite as you go

Keep one row per fact and replace it. The moment a customer asks "why did it think that?", there is no answer left to give.

All three share a failure: **they store text, not knowledge**. None of them can say what is currently true about a person, how confident it is, or what it believed last month.

Two calls. Everything else is our problem.

```
// after the user speaks
await memora.remember({
  userId, content: userMessage
});

// before you answer
const { results } = await memora.recall({
  userId, query: userMessage
});
```

Send what someone said. Ask for what matters now. Memora decides what was worth keeping, notices when it changes, and hands back the few facts that bear on this moment — each one carrying the reason it was returned.

It extracts, not archives

Durable facts, preferences, goals and decisions — not transcripts.

It versions, never overwrites

A changed fact links to the one it replaced, with the reason it did.

It surfaces contradictions

When two things can't both be true, both come back flagged. It does not pick a side quietly.

It explains every answer

Similarity, confidence, freshness, and a sentence a human can read.

The category leader made a choice that leaves a door open.

In April 2026, mem0 – 63.7k GitHub stars, the default answer for agent memory – rebuilt its algorithm around append-only extraction. When a fact changes, the new one lives beside the old and conflicts are resolved silently at read time. It is faster, and it is a real engineering result.

It also means their architecture cannot tell you why it believes something. There is no stated reason, no flagged conflict, no version with an explanation attached. For a consumer chatbot that is a fine trade. For anyone who has to answer to a regulator, a customer, or an auditor, it is disqualifying.

Memora keeps the judgement and moves it off the response path – same speed, full audit trail. **"Memory your compliance team can read"** is a sentence they architecturally cannot say.

75%

LoCoMo, first run on data never tuned against

100%

On our own regression set (14/14)

17/17

End-to-end through the real HTTP surface

\$9.87

Model cost per 1,000 memories

LOCOMO CATEGORY	SCORE	WHAT IT TESTS
abstain	100% (5/5)	Refusing to answer what it was never told — the hallucination check
open-domain	100% (1/1)	Inference beyond what was stated
single-hop	71% (5/7)	One fact, directly recalled
multi-hop	67% (2/3)	Combining facts from different conversations
temporal	50% (2/4)	When something happened — our weakest area, and we know why

mem0 publishes 92.5 on the full LoCoMo set using partial-credit F1 scoring. Ours is a 20-question stratified sample graded strictly by a judge model, so the two are **not directly comparable** — and we will say so until they are. Three of our five misses were retrieval simply not surfacing the fact, which is a fixable ranking problem, not a reasoning one.

Sell to the teams for whom "why did it say that?" has a legal answer.

01 • SUPPORT

Customer support platforms

An agent that remembers a customer's history across tickets, and an audit trail when the customer disputes what it was told.

02 • FINTECH

Financial services

Suitability and KYC facts change.
Regulators ask what you knew and when.
Versioned memory with stated reasons is the answer.

03 • HEALTH

Health & care

An allergy contradicted by a later statement must be flagged for a human, never silently resolved. That is our default behaviour.

Same product underneath. The wedge is that in these three markets, the audit trail is not a feature — it is the reason procurement signs.

Metered on memories written, because that is what costs us money.

PLAN	WRITES / MONTH	READS / MONTH	RATE LIMIT	STATUS
Free	1,000	10,000	60/min	Enforced today
Starter	25,000	250,000	300/min	Enforced today
Pro	250,000	2,500,000	1,200/min	Enforced today
Enterprise	unlimited	unlimited	6,000/min	Enforced today

Every request is already checked against scope, rate limit and monthly quota, and every model call is metered per project and per key. **The enforcement is built. The prices are not set** – the one number we have is \$9.87 of model spend per 1,000 memories, and whatever we charge has to sit above it with room.

Reads are cheap for us and valuable to customers, so every tier allows ten reads per write. That ratio is deliberate: it rewards the behaviour we want – recall before every answer.

The engine is finished. The business around it is not.

BUILT AND PROVEN

- ✓ **Memory engine** — extraction, reconciliation, versioning, contradictions
- ✓ **Hybrid retrieval** — semantic and keyword search, rank-fused
- ✓ **Dashboard** — browse, filter, evidence trails, version timelines
- ✓ **SDK and MCP server** — Claude Code and Cursor can use it directly
- ✓ **Guards** — key scopes, rate limits, quotas, idempotency
- ✓ **Eval harness** — every claim on this page is reproducible

NOT YET

- ✗ **Not deployed** — runs locally; no hosting, no domain, no backups
- ✗ **No billing** — metering is done, nothing charges anyone
- ✗ **No team accounts** — single owner, no invites or password reset
- ✗ **No alerting** — failures land in logs, nobody is paged
- ✗ **No legal** — ToS, privacy policy and DPA are not optional here
- ✗ **No customers** — nobody has used this but us

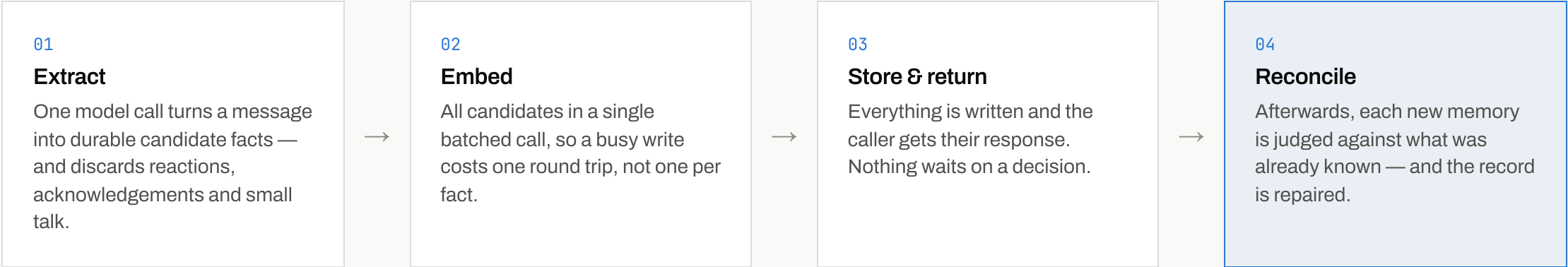
Two to four weeks of unglamorous work stands between this and a first paying user. **None of it is research.**

PART TWO

How it works, and what it still needs

The mechanism, the integration, and an honest list of everything missing.

Writes return immediately. Judgement happens behind the response.



VERDICT	WHAT HAPPENS TO THE RECORD
ADD	Genuinely new. Nothing to repair.
UPDATE	Links the new version to the old one and retires the old, with the reason it changed.
MERGE	Folds added detail into a single combined fact.
IGNORE	A restatement — refreshes the original's confidence instead of duplicating it.
FLAG	Cannot both be true. Both memories are marked unresolved and surfaced to a human.

HOW IT WORKS • WRITING

Two searches, fused — and an answer that shows its working.

Meaning and words, together

Embeddings find what a question is about; a full-text index finds the exact strings embeddings blur — cluster names, ticket ids, channel handles. The two rankings are fused, so a fact found either way still surfaces.

Ranked on more than similarity

Relevance is 60% match, 25% how confident we are the fact is true, 15% how recently it was confirmed. An old, shaky memory loses to a fresh, certain one.

History travels with the fact

Superseded versions do not compete for a slot in the ranking, but they ride along — so "where do they work?" and "did they ever work at X?" are both answerable.

```
// what comes back
{
  content: "Is allergic to peanuts",
  type: "fact",
  status: "active",
  relevanceScore: 0.87,
  similarity: 0.84,
  confidence: 0.95,
  freshness: 0.99,
  matchedOn: "both",
  reason: "high semantic match (0.84),
          high confidence,
          recently confirmed",
  history: []
}
```

Every field is there so an application — or an auditor — can see why this memory and not another one.

1 · Install and write

```
npm install @memora/client

import { Memora } from "@memora/client";
const memora = new Memora({ apiKey });

await memora.remember({
  userId: "user_123",
  content: "I'm vegan and allergic to peanuts."
});
```

2 · Recall before you answer

```
const { results } = await memora.recall({
  userId, query: userMessage, topK: 5
});

const system = results
  .map(m => `- ${m.content}`)
  .join("\n");
```

Or skip the code entirely

An MCP server ships with it. Point Claude Code or Cursor at Memora and the agent gets remember, recall, and the ability to record what it learned from a failed task.

Ask why, any time

`explain(memoryId)` returns the full evidence trail: every source excerpt, every version, and the stated reason for each change.

Narrow it down

Filter recall by memory type, minimum confidence, time window, agent or session — so a support bot and a sales bot never read each other's notes.

Environments, not guesswork

Every key is scoped to one environment. A development key cannot read production memories. Ever.

Deliberately boring infrastructure.

STORAGE

One Postgres

Memories, embeddings, full-text index, job queue, rate limits and idempotency all in the same database. No Redis, no vector service, no second thing to page someone about.

INTELLIGENCE

Claude + Voyage

Claude extracts and judges; Voyage embeds. Both are swappable behind an interface — a requirement for enterprises on Bedrock or Azure.

RUNTIME

Stateless app

Next.js in a 290 MB container, non-root, migrations baked in. Scale by adding instances; limits stay correct because they are counted in the database.

DURABILITY

A real queue

Reconciliation jobs are rows, not promises. A crash, a deploy or a failed model call means a retry — never a memory that silently never got judged.

Every architectural choice here was made to reduce the number of things that can be quietly wrong at 3am. **The audit trail is the product; losing part of it is not an acceptable failure mode.**

Nothing on this list is research. All of it is required.

NEEDED	WHY IT BLOCKS	WHO
BLOCKER Voyage payment method	The embeddings account is capped at 3 requests per minute. The product cannot serve real traffic, and no latency figure is meaningful until it is lifted.	You
BLOCKER Hosting	The container and compose file are built and boot-tested. There is no server, domain, TLS or managed database to put them on.	You
BLOCKER Billing	Usage is metered per model call and per request — the hard half. Stripe, plan upgrades and payment failure handling are not built.	Both
SERIOUS Team accounts	One owner per organisation. No invites, roles, password reset or email verification — every one of which a buyer will ask for.	Us
SERIOUS Observability	Failures land in container logs. No error tracking, no alerting, no runbook for a customer-visible incident.	Us
SERIOUS Legal & privacy	We would be storing personal data about other companies' users. ToS, privacy policy, DPA, PII redaction and an audit log are prerequisites, not polish.	You
NEXT Retrieval at scale	A dense conversation produced 210 memories for one user; three of five benchmark misses were the right fact never being retrieved. Ranking, not reasoning.	Us

Four weeks to a first paying customer.

WEEK 1

Unblock and deploy

Voyage billing. Managed Postgres, a domain, TLS, backups with a tested restore. The image already boots; it needs somewhere to boot.

WEEK 2

Charge for it

Stripe on top of the existing metering, plan upgrades, dunning. Team invites, roles, password reset.

WEEK 3

Make it observable

Error tracking, alerting on failed reconciliation, a runbook. Publish the SDK and MCP server so people can actually install them.

WEEK 4

Prove it harder

Full LoCoMo and LongMemEval rather than a sample. Fix retrieval at scale. Legal pack. Then onboard the first design partner.

The riskiest thing on this timeline is not technical. **It is that nobody outside this room has used it yet** — which is why week four ends with a design partner, not a launch.

Three decisions, and one of them changes everything below it.

DECISION 01

Open source, or not?

mem0's 63.7k stars are a distribution engine we cannot out-run from a closed repo. An Apache-2 core with a hosted tier is the standard answer — and it gives away the engine. This decision shapes every other one.

DECISION 02

Is Experience Memory its own product?

Agents recording what they learned from failed tasks, and being advised by it before the next attempt. It is the one thing mem0 has no equivalent of, and today it is a tab in a dashboard rather than a thing with a name and a price.

DECISION 03

Which vertical goes first?

Support, fintech or health. The answer sets the docs, the examples, the compliance work and the first ten conversations. Picking one beats being generically available to all three.

Everything else on the roadmap is work we can simply do. These three are yours, and until they are answered the sequencing is a guess.



Storage is table stakes. Knowing why is the product.

Every number in this deck is reproducible: `pnpm test`, `pnpm eval`, `pnpm smoke`.